Mobile Cellular Communications (5G)

PART II
5G networks

I. Objectives

The objectives of this laboratory are:

- Identify configuration parameters required for the different components
- Understand the main procedures in a mobile cellular (5G) at the control and data planes by running opensource implementations of the main components

II. Duration

This laboratory should last 4h30, divided in 3 classes.

III. Used tools

This laboratory will use:

a) An opensource implementation of a 5G Core: Free5GC [free5GC] [free5gcwiki]

b) A gNB and UE opensource implementation: UERANSIM [ueransim]

c) A VirtualBox VM with both components already installed in the laboratory PCs

d) Wireshark also installed in the laboratory PCs

The VM is also available via SSH at port 2222 for user '***ubuntu***' (e.g. '***ssh -p 222 ubuntu@localhost***', from the hosting machine); password is '***ubuntu***' for users '***ubuntu***' and ***root***.

IV. Network diagram

1. 5G Core components (provided by Free5GC) are represented by light blue (control plane) and purple boxes (UPF, dataplane), gNBs by ligth orange boxes, and UEs by green boxes
2. With UERANSIM, the 5G-NR radio interface ('Radio Link', RL) is emulated over UDP between the UEs (11, 12 and 21, green boxes) and the gNBs (gNB1 and gNB2, light orange boxes) they are connected to.
3. IP addresses:
   a. 10.0.123.0/24 (red): SBI; Core components, Web Console and DB (control plane)
   b. 10.0.124.0/24 (orange): N2 interfaces (control plane)
   c. 10.0.130.0/24 (purple): N3 interfaces (data plane)
   d. 10.0.140.0/24 (dark red): N4 interfaces (control plane)
   e. 10.0.20[1|2].0/24 (green): radio interfaces emulation
   f. 10.1.[1|2].0/24 (blue): N6 DNNs (data plane)

4. Via the 'Host' entity, emulated UEs can reach the Internet.
5. A ***hosts*** file has been added to Wireshark (***/root/.config/wireshark***) for IP addresses resolution so that Wireshark presents components' names instead of IP addresses, allowing a faster and better interpretation of the messages exchange (see that file contents in Annex F at the end).
6. The shown MongoDB in the diagram component serves as persistent data repository for the other components while the network is running (you will see a significant volume of messages with it).

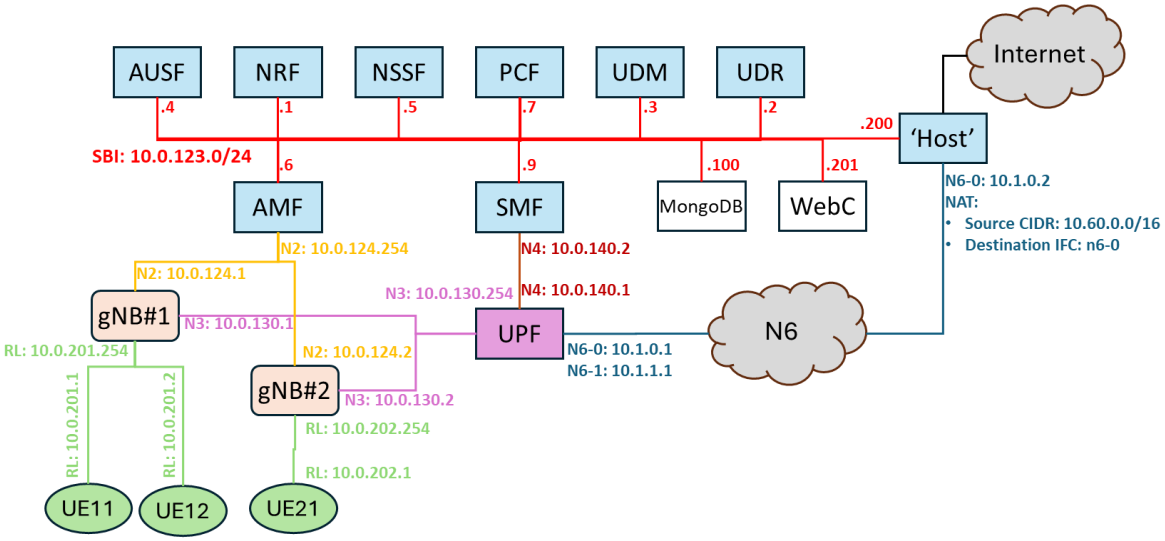


Figure 1: Network diagram

V. Procedures

1. Introduction

During the laboratory execution, based on the provided VirtualBox Virtual Machine, the 5G network components will be started and stopped, following this order (numbers in parentheses indicate the guide execution step):

1. **5G Network: (a) 5G Core (3.7) → (b) gNB1 (4.3) → (c) gNB2 (4.5) →**
2. **UEs creation: → (d) UE provisioning at the 5G Core (5.2) →**
3. **UEs start: → (e) UE11 (5.4) → (f) UE12 (6.5) → (g) UE21 (6.7) →**
4. **Stop the system: → (h) UEs, gNBs and 5G Core (9.1 and 9.3)**

Linux *Namespaces* are used to run each of the nine 5GC Network Functions (AMF, AUSF, NRF, NSSF, PCF, SMF, UDM, UDR, UPF) inside its own namespace [konrad]. This allows the usage of Wireshark (**shall be started with 'sudo'**) to capture traffic packets exchanged between any two NFs, on their own interfaces (you will get the list of interfaces present in the following screen capture, when selecting capturing interfaces in Wireshark, after 5G Core components have been started).

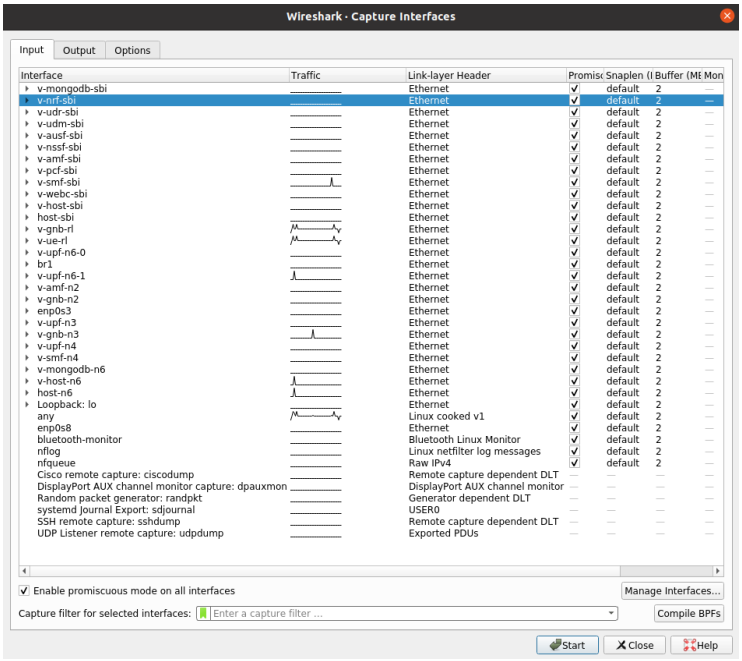


Figure 2: Logical interfaces as seen in Wireshark, after 5G Core start

2. Configurations analysis

1) Analyse the yaml configuration files in the list below (2.1.a), located in folder *~/5GLab/netns5g/config* (you may open them with the File Manager) and search for the listed configuration parameters in 2.1.b.

   a. **Files**:

      i. 5G Core: **amfcfg**.yaml, **smfcfg**.yaml, **upfcfg**.yaml

      ii. 5G RAN: free5gc-**gnb1**.yaml, free5gc-**gnb2**.yaml

      iii. 5G UEs:

      - free5gc-**ue11**.yaml
      - free5gc-**ue12**-**sl1**.yaml, free5gc-**ue12**-**sl2**.yaml
      - free5gc-**ue21**.yaml

```
ubuntu@ubuntu-VirtualBox:~/5GLab/netns5g/config$ more amfcfg.yaml
info:
  version: 1.0.3
  description: AMF initial local configuration

configuration:
  amfName: AMF # the name of this AMF
  ngapIpList:  # the IP list of N2 interfaces on this AMF
    - 10.0.124.254
  sbi: # Service-based interface information
    scheme: http # the protocol for sbi (http or https)
    registerIPv4: 10.0.123.6 # IP used to register to NRF
    bindingIPv4: 10.0.123.6  # IP used to bind the service
    port: 8000 # port used to bind the service
    tls: # the local path of TLS key
      pem: config/TLS/amf.pem # AMF TLS Certificate
      key: config/TLS/amf.key # AMF TLS Private key
  serviceNameList: # the SBI services provided by this AMF, refer to TS 29.518
    - namf-comm # Namf_Communication service
    - namf-evts # Namf_EventExposure service
    - namf-mt   # Namf_MT service
    - namf-loc  # Namf_Location service
    - namf-oam  # OAM service
  servedGuamiList: # Guami (Globally Unique AMF ID) list supported by this AMF
    # <GUAMI> = <MCC><MNC><AMF ID>
    - plmnId: # Public Land Mobile Network ID, <PLMN ID> = <MCC><MNC>
        mcc: 001 # Mobile Country Code (3 digits string, digit: 0~9)
        mnc: 01 # Mobile Network Code (2 or 3 digits string, digit: 0~9)
      amfId: cafe00 # AMF identifier (3 bytes hex string, range: 000000~FFFFFF)
  supportTaiList:  # the TAI (Tracking Area Identifier) list supported by this AMF
    - plmnId: # Public Land Mobile Network ID, <PLMN ID> = <MCC><MNC>
```

Figure 3: Example of 5G Core entity (AMF) configuration file (partial)

```
ubuntu@ubuntu-VirtualBox:~/5GLab/netns5g/config$ more free5gc-gnb1.yaml
mcc: '001'          # Mobile Country Code value
mnc: '01'           # Mobile Network Code value (2 or 3 digits)

nci: '0x000000010'  # NR Cell Identity (36-bit)
idLength: 32        # NR gNB ID length in bits [22...32]
tac: 1              # Tracking Area Code

linkIp: 10.0.201.254   # gNB's local IP address for Radio Link Simulation (Usually same with local IP)
ngapIp: 10.0.124.1   # gNB's local IP address for N2 Interface (Usually same with local IP)
gtpIp: 10.0.130.1    # gNB's local IP address for N3 Interface (Usually same with local IP)

# List of AMF address information
amfConfigs:
  - address: 10.0.124.254
    port: 38412

# List of supported S-NSSAIs by this gNB
slices:
  - sst: 0x1
    sd: 0x010203
  - sst: 0x2
    sd: 0x112233

# Indicates whether or not SCTP stream number errors should be ignored.
ignoreStreamIds: true
```

Figure 4 Example of 5G RAN (gNB) configuration file (partial)

```
ubuntu@ubuntu-VirtualBox:~/5GLab/netns5g/config$ more free5gc-ue11.yaml
# IMSI number of the UE. IMSI = [MCC|MNC|MSISDN] (In total 15 digits)
supi: 'imsi-001010000000011'
# Mobile Country Code value of HPLMN
mcc: '001'
# Mobile Network Code value of HPLMN (2 or 3 digits)
mnc: '01'

# Permanent subscription key
key: '8baf473f2f8fd09487cccbd7097c6862'
# Operator code (OP or OPC) of the UE
op: '8e27b6af0e692e750f32667a3b14605d'
# This value specifies the OP type and it can be either 'OP' or 'OPC'
opType: 'OPC'
# Authentication Management Field (AMF) value
amf: '8000'
# IMEI number of the device. It is used if no SUPI is provided
imei: '356938035643811'
# IMEISV number of the device. It is used if no SUPI and IMEI is provided
imeiSv: '4370816125816111'

# List of gNB IP addresses for Radio Link Simulation
gnbSearchList:
  - 10.0.201.254

# UAC Access Identities Configuration
uacAic:
  mps: false
  mcs: false

# UAC Access Control Class
uacAcc:
  normalClass: 0
  class11: false
  class12: false
  class13: false
  class14: false
  class15: false

# Initial PDU sessions to be established
sessions:
  - type: 'IPv4'
    apn: 'internet'
    slice:
      sst: 0x1
      sd: 0x010203
```

Figure 5: Example of 5G User Equipment configuration file (partial)

b. **In those files, search for and identify the following parameters**:

   i. MCC and MNC

   ii. NR Cell Identities and TACs

   iii. Supported slices at gNB1 and gNB2 (SST+SD)

   iv. Supported DNN

   v. List of SUPIs (UE11, UE12 and UE21)

c. **For entities in the SBI (e.g. AMF) check other parameters and information**:

   i. IP addresses matching with the figure above

   ii. Exposed services (e.g. Namf***)

3. 5GC start

2) Open a terminal window
3) Change to directory (~/5GLab/netns5g) containing the scripts needed to setup and run the 5G environment
4) Initialize environment (create the namespaces and the virtual interfaces)

```
~/5GLab/netns5G$ sudo ./5Gsetup.sh
```

5) Check created namespaces and connecting links

```
~/5GLab/netns5G$ sudo ip netns # lists created namespaces
~/5GLab/netns5G$ sudo ip link # lists created links
```

6) Start a Wireshark

```
~/5GLab/netns5G$ sudo wireshark &
```

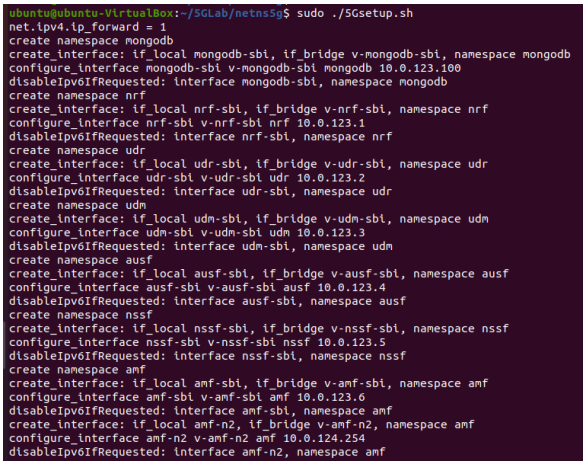


Figure 6: start of namespaces (not complete)

7) Start the capture in the interface 'br1' (this will capture all the traffic; you can start other Wireshark instances at specific interfaces, e.g. 'v-amf-sbi' to see only that specific traffic)
8) Start the 5G Core (free5gc)

```
~/5GLab/netns5G$ sudo ./5Gstart.sh
```

Check there are no errors reported in shown messages; if no, check for a potential solution in Annex G

When seeing 'all services are running', 5G Core Network Functions (NFs) have started, each in its own namespace.

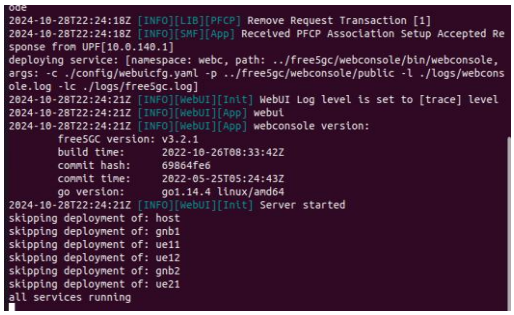
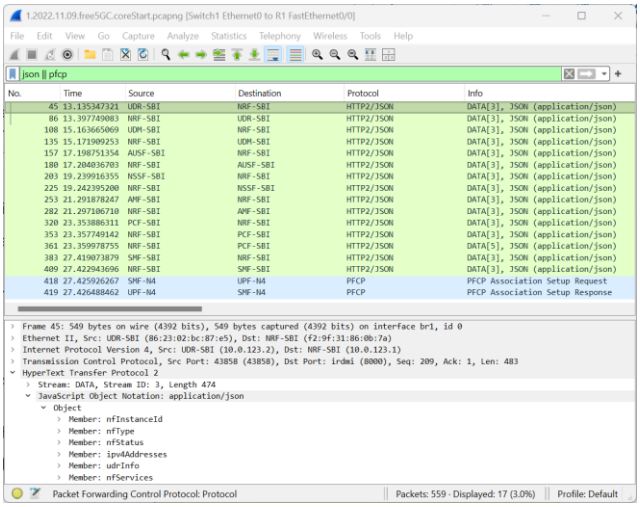


Figure 7: start of 5G Core (last messages)

a. With 'ps -gaxu | grep free5gc' check the different running processes and the order they started (see the processes ID)
b. Observe the script output and register, confirming the order by which 5G Core components have been started. Relate the order they appear with the existing inter dependencies.
c. Observe the successive interactions with NRF; what is that for?

9) Stop the capture and identify the involved protocols (to facilitate it, order the capture by the 'Protocol' column by pressing the respective column top); which of those are specific 5G Core protocols?
   a. Complementary, go to 'Statistics' → 'Conversations'; check 'Name resolution' and analyse the results
   b. In addition, go to 'Statistics' → 'Protocol Hierarchy' and identify here the previously identified protocols

10) Identify the dialogs for identified 5G protocols; apply a display filter for those protocols and check the involved entities



    a. Go to 'Statistics' → 'Flow Chart'; observe the sequence of the interactions in the diagram

    b. Filter only by 'json':

        i. Observe the pairs of messages; With which other entity are these talking to? What for?

        ii. UPF does not appear; why?

        iii. Analyse the protocol stack for the exchanged JSON messages

        iv. In one of the exchanged messages, expand the subtrees under JSON; briefly observe the contents of several of these messages

    c. Filter only by 'pfcp':

        i. Which entities are here involved? Observe the message types

        ii. Analyse the protocol stack for PFCP

11) Open and briefly analyse the log files present in ~/5Glab/netns5G/logs for the different entities; SMF example:

```
time="2025-11-04T21:03:25Z" level=info msg="Endpoints: [10.0.130.254]" category=CTX component=SMF
time="2025-11-04T21:03:25Z" level=info msg="Server started" category=Init component=SMF
time="2025-11-04T21:03:25Z" level=info msg="SMF Registration to NRF {071cf68e-68bb-4c6e-b9e1-1060c0b3ad60 SMF REGISTERED 0 0xc00009c2e0 0xc00009c320 [] []  [10.0.123.9] [] <nil> [] [] <nil> 0 0 0 area1 <nil> <nil> <nil> <nil> 0xc0000ae240 <nil> <nil> <nil> <nil> map[] <nil> false 0xc00009c120 false false []}" category=Init component=SMF
time="2025-11-04T21:03:25Z" level=info msg="Listen on 10.0.140.2:8805" category=PFCP component=SMF
time="2025-11-04T21:03:25Z" level=info msg="Sending PFCP Association Request to UPF[10.0.140.1]" category=App component=SMF
time="2025-11-04T21:03:25Z" level=info msg="Remove Request Transaction [1]\n" category=PFCP component=LIB
time="2025-11-04T21:03:25Z" level=info msg="Received PFCP Association Setup Accepted Response from UPF[10.0.140.1]" category=App component=SMF
```

4. gNBs start

12) Open a (new) terminal window/tab

13) Start Wireshark instance and start capturing in interface br1 (do not stop Wireshark until the end of this section, in step 18)

    `$ sudo wireshark`

    'Capture' → 'Options' → select 'br1'

14) At the same directory as for the 5G Core, start the first gNB (gNB1)

    `~/5GLab/netns5G$ ./GNB1start.sh`

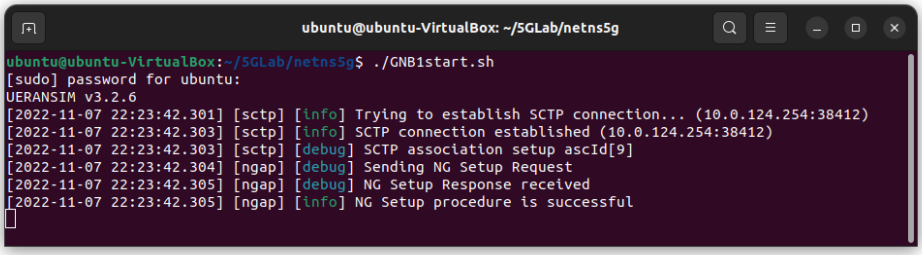


Figure 8: GNB start log

15) In the live Wireshark capture, observe/note the following:

    a. Check the output messages of both gNB1 (previous figure) and the 5GCore (AMF)

    b. Repeat the identification of the involved protocols and the specific 5G ones (suggestion: use 'Statistics' → 'Protocol Hierarchy' and 'Conversations')

    c. The SCTP connection setup and later the exchanged heartbeats (suggestion: filter the displayed packets by identified 5G protocols)

    d. Identify the involved entities

    e. Detail to the maximum extent, in the Packet Details window, the *NGsetupRequest* and *NGsetupResponse* messages (with mouse right button in '*Packet Details*', select '*Expand Subtrees*'); Confirm observed values with the ones obtained from the configuration files analysis

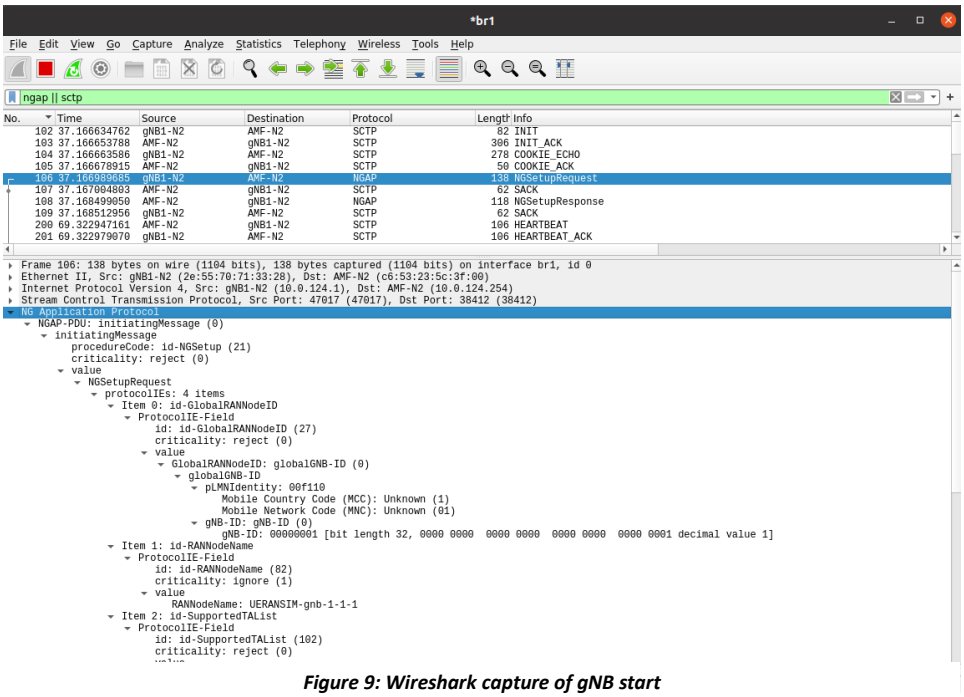


Figure 9: Wireshark capture of gNB start

16) Observe the periodic SCTP heartbeats

17) Start the second gNB (gNB2)

```
~/5GLab/netns5G$ ./GNB2start.sh
```

18) In the live Wireshark **compare** the new *NGsetupRequest* and *NGsetupResponse* messages with previous ones (gBN1) (suggestion: apply a display filter for the NGAP protocol only and order the capture by the 'Info' column and then move the two pairs of captured packets)

19) **Observe the logs** in the screen (Core, gNB1 and gNB2) and logfiles in: ~/5GLab/netns5g/logs (suggestion: use the 'Files' application to see and open the most recent files, the ones generated until now, executing this 5G Lab)

5. UE creation, registration and default PDU creation

20) Open the Free5GC Web Console from the web browser:
    a. ***http://10.0.123.201:5000***
    b. credentials: '***admin***'/'***free5gc***'

21) Create the 3 UEs from the table below ('**Subscribers**' → '***New Subscriber***'; see screen capture in Annex D):

	UE11	**UE12**	**UE21**
PLMN ID (MCC/MNC)	001 01	001 01	001 01
SUPI (MSIN)	001 01 0000 0000 **11**	001 01 0000 0000 **12**	001 01 0000 0000 **21**
SST/SD	1/010203 (sl1)	1/010203 (sl1) 2/112233 (sl2)	1/010203 (sl1)
DNN	internet	internet	internet
UL/DL AMBR	10/20 Mbps	100/200 Mbps	1/2 Mbps
5QI	9	9	9
Note	Will connect to **gNB1**	Will connect to **gNB1**	Will connect to **gNB2**

Notes: **Only change** the parameters shown in the table and only if required (if ones in the console are different from the ones in the table)

1. **do not change**: Authentication method, K*, Operator Code Type, Operator Code Value*, and SQN*)
    a. In this laboratory K and OPc are the same for all UEs, contrary to a real network
2. you may search and interpret the other parameters.

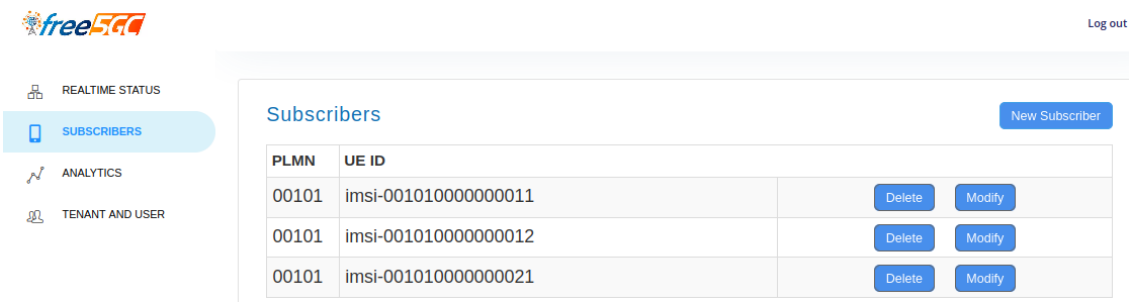


Figure 10: free5GC WebGUI after creation of the 3 subscribers

22) Check these values against the ones present in the UEs' .yaml configuration files (e.g. 'free5gc-ue11.yaml')
    a. SUPI, MCC, MNC, key (K), OP(c), SST/SD

23) In the Free5GC "*New Subscriber*" form, for UE11 and UE21, delete the second appearing S-NSSAI (*Single Network Slice Selection Assistance Information*) and the second DNN ('internet2')

24) After creating the 3 UEs (11, 12 and 21) go to 'Realtime Status' and observe the absence of information

25) Restart the Wireshark capture, keeping the capture in the same interface ('br1')

26) Start the first UE (UE11)

```
~/5GLab/netns5G$ ./UE11start.sh
```

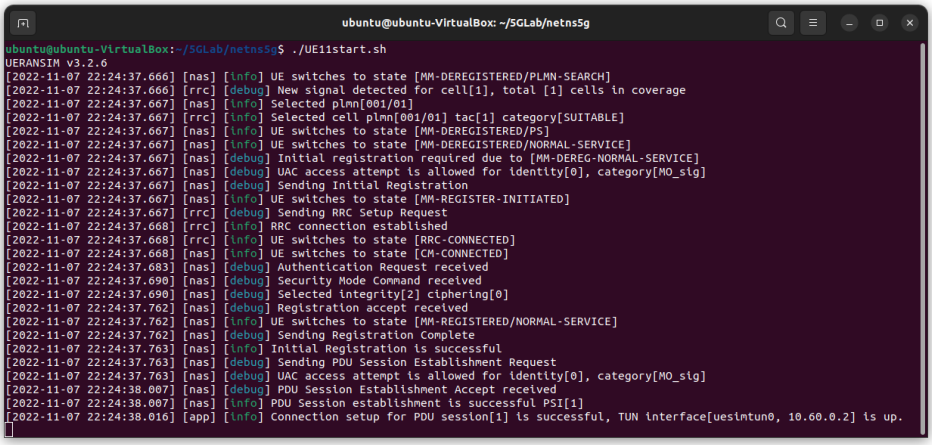


Figure 11: log of UE11 start

27) Observe the indicated states the UE went through, during the process, in the produced output; observe the other messages and its sequence

28) Observe the creation of the new TUN interface ('uesimtun0'); in a new terminal window, you can check the creation of this interface in namespace 'ue11' and note its IP address (10.60.0.2, in the example)

```
~/5GLab/netns5G$ sudo ip netns exec ue11 ip addr
```

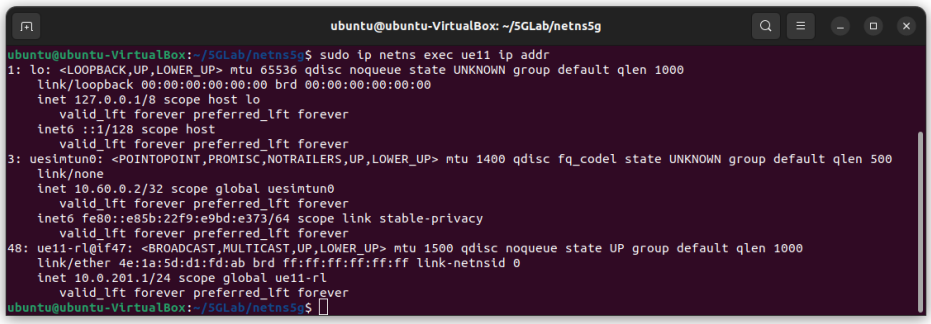


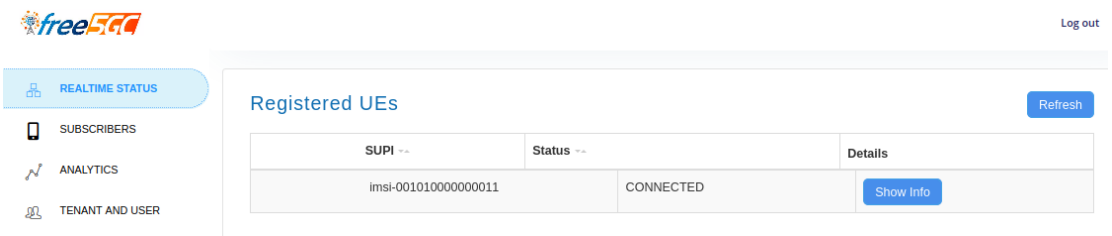
Figure 12: check UE11 assigned IP addresses

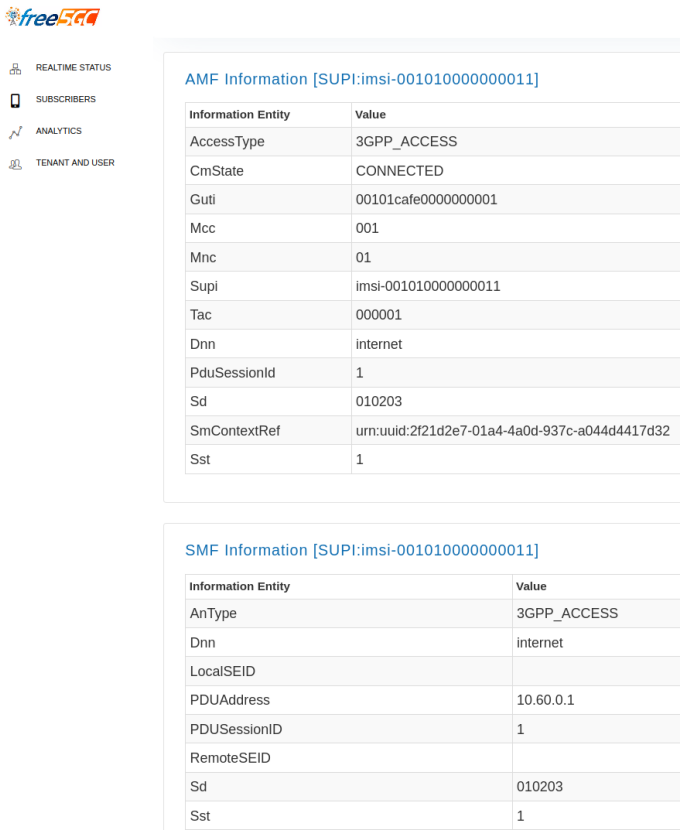
This is the interface that will be used to exchange traffic via the 5G network.

a. Order the Wireshark displayed capture by 'Protocol' (press the respective column name) and list the relevant protocols

b. Observe in the SMF configuration file ('smscfg.yaml') the IP pools definition for slice 1/010203 and slice 2/112233 to understand the IP address assigned to UE11.

29) Go again to the Web Console, 'Realtime Status' and observe its contents; click on 'Show info' for UE11 and analyse its contents.





30) Go to ‘Statistics’ and select ‘Conversations’ in Wireshark, order by ‘IPv4’ and enable ‘Name resolution’; order by ‘Rel Start’ and observe the 5G dialogs; you may also order by ‘Address A’ and ‘Address B’

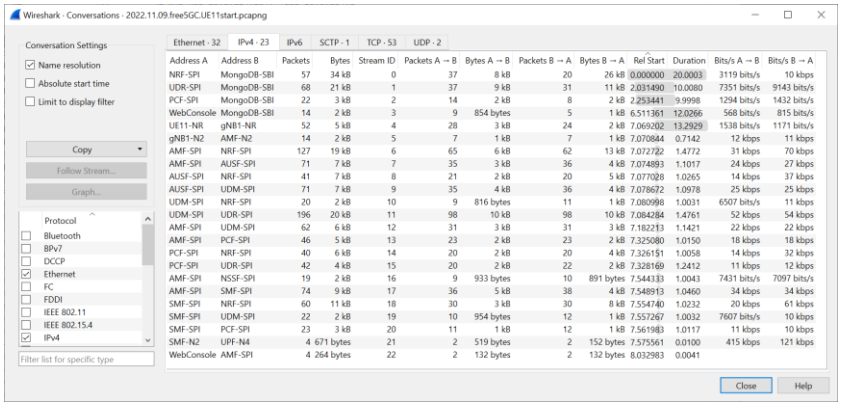


Figure 13: interactions analysis for UE11 start

31) Apply a Display Filter to see just NGAP protocol ("***ngap***")
    a. Identify the involved 5G control functions (IP addresses are already translated to the functional entity interface, according to the diagram above); identify the dialog between gNB1, AMF and SMF and their sequence (check against the class slides)
        i. If adding 'udp' to the filter, you will also see the UE dialogs but unable to decode (the interface is emulated over UDP, as stated)
    b. Observe the sequence of exchanged messages, looking into their details in the Packet Details window;
        i. 'Expand Subtrees' below 'NG Application Protocol', for the following messages:
            1. Registration Request, Authentication Request and Authentication response
        ii. Besides other parameters, search for: RES, RAND, SQN and MAC
32) Add ("**ngap || pfcp**") to the visualization filter and check where these messages are located
    a. Expand the messages 'Session Establishment Request'
        i. Observe the contents of the *Information Elements* (IE) Create PDR, FAR and QER
        ii. FAR and QER IDs present in the PDRs and the FAR and QER IE present in the message
        iii. Observe TEID values
        iv. Actions (e.g 'Forward' in FAR) and involved interfaces (e.g. 'N6-LAN')
    b. Expand the message 'Session Modification Request'
        i. Do the same exercise as for previous message
        ii. Check for the differences and justify this message
33) Add "**or json**" to the filter to see the complete dialogs that took place in SBI; specifically apply the filter "***ngap || pfcp || json && (!(ip.addr==10.0.123.1) && !(ip.addr==10.0.123.2) && !(ip.addr==10.0.123.3))***", to see the dialogs at N1, N2, and SBI, excluding NRF, UDR and UDM (you may need to add other IP addresses, to end up with just AMF, SMF and PCF) – see next figure
    a. To help and complement the analysis go to 'Statistics' and 'Flow Graph'; check the 'Limit to display filter' is on
    b. In this window, if you select a message it is immediately identified in the main capture window
34) Expand the 'InitialContextSetupRequest' message sent from the AMF to the gNB1 and observe all the indicated parameters
35) Expand the 'UpLinkNASTransport' message sent from the gNB1 to the AMF and observe all the indicated parameters
36) Expand the 'PDU session establishment request' message sent from the AMF to SMF and observe all the indicated parameters
37) Filter the conversation between the SMF and PCF and see the exchanged parameters

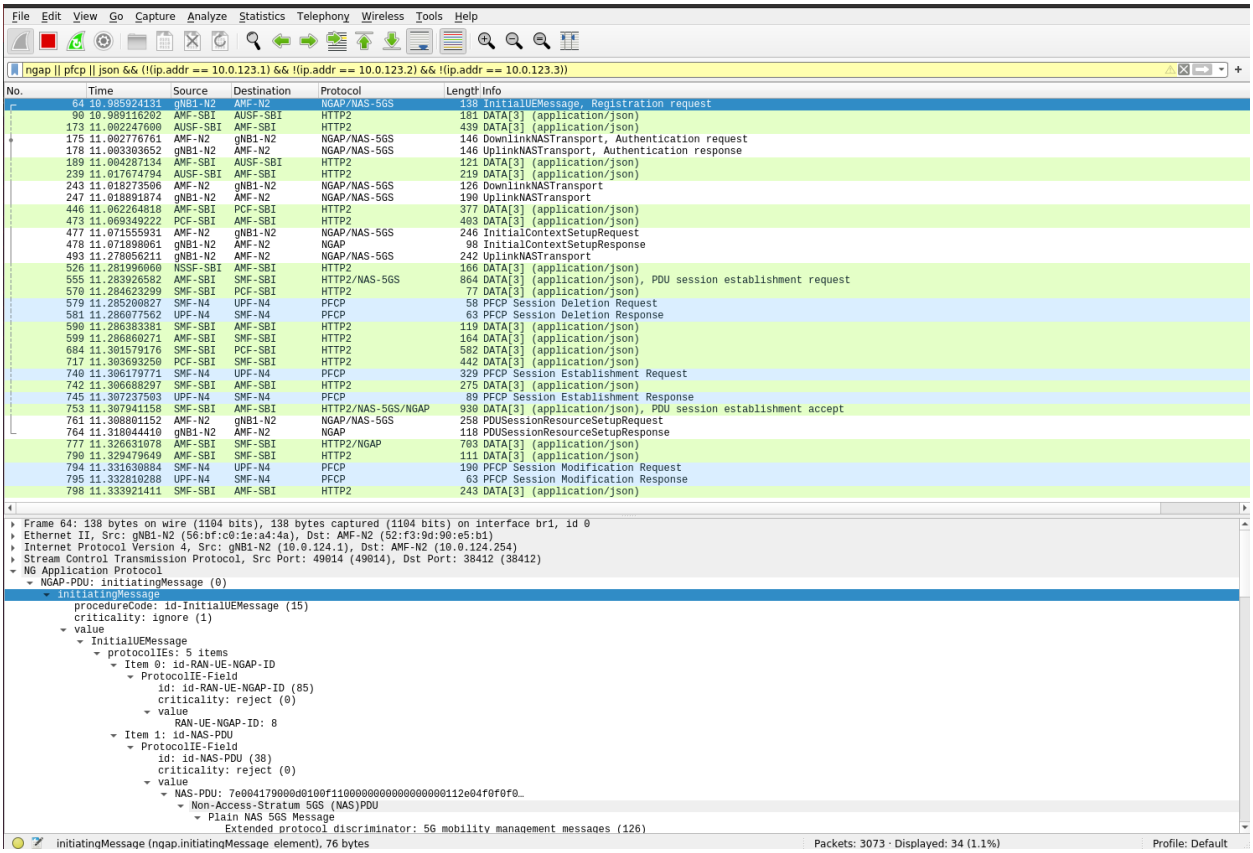


Figure 14: Parcial set of captures messages at UE11 start

6. Connectivity

38) Start a Wireshark capture in the interface ‘upf-n3’ and another capture in the interface ‘upf-n6-0'

39) Apply a Display Filter to see protocols GTP and ICMP

40) In a terminal window, start a ping to 8.8.8.8 from UE11

```
~/5GLab/netns5G$ sudo ip netns exec ue11 ping 8.8.8.8 -I uesimtun0
```

```
ubuntu@ubuntu-VirtualBox:~$ sudo ip netns exec ue11 ping 8.8.8.8 -I uesimtun0
PING 8.8.8.8 (8.8.8.8) from 10.60.0.1 uesimtun0: 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=113 time=17.8 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=113 time=17.7 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=113 time=17.5 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=113 time=17.7 ms
^C
--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3019ms
rtt min/avg/max/mdev = 17.546/17.689/17.773/0.085 ms
ubuntu@ubuntu-VirtualBox:~$
```

Figure 15: UE11 working ping to an external test IP (8.8.8.8 as example)

41) Analyse, in the Wireshark Packet Details, the GTP encapsulation

    a. Observe the ICMP/IP/GTP/UDP/IP encapsulation and the used ports and IP addresses

    b. Observe the *Tunnel Endpoint IDentifier* (TEID) in both directions of the communication

42) In a new Terminal Window/Tab, Start UE12 in slice 1 (SSD/SD: 1/010203)

```
~/5GLab/netns5G$ ./UE12start-sl1.sh
```

(check the contents of file *./config/free5gc-ue12-sl1.yaml*)

43) Make a ping from UE11 to UE12

```
~/5GLab/netns5G$ sudo ip netns exec ue11 ping <U12 IP addr> -I uesimtun0
```

a. Maintain or apply a filter for ICMP (or GTP)

b. Observe and justify the apparent duplication of these packets

c. Observe the different used TEIDs

d. What is the path, in terms of 5G entities, for the exchanged ICMP packets between UE11 and UE12? Make the analysis based on IP addresses of the outer IP packets.

e. Being both connected to gNB#1, are the packets going to the UPF? Why?

44) In a new Terminal Window/Tab, Start UE21

```
~/5GLab/netns5G$ ./UE21start.sh
```

(check the contents of file *./config/free5gc-ue21.yaml*)

45) Make a ping from UE12 to UE21

```
~/5GLab/netns5G$ sudo ip netns exec ue12 ping <U21 IP addr> -I uesimtun0
```

46) What is the path, in terms of 5G entities, for the exchanged ICMP packets between UE12 and UE21?

a. Make the analysis based on IP addresses of the outer IP packets

7. QoS

47) Open a **new terminal** window and **start an iperf3 server** at the DNN

```
$ iperf3 -s
```

48) Check the TUN interface name and assigned IP address

```
$ sudo ip netns exec ue11 ip addr
```

49) Start an iperf3 client at UE11 towards the server instance and register the achieved bandwidth in the UL and DL directions

```
$ sudo ip netns exec ue11 iperf3 -c 10.1.0.2 -B <ue11 IP address> -- uplink
$ sudo ip netns exec ue11 iperf3 -c 10.1.0.2 -R -B <ue11 IP address> -- downlink
```

```
ubuntu@ubuntu-VirtualBox:~/5GLab/netns5G$ sudo ip netns exec ue11 iperf3 -c 10.1.0.2 -B 10.60.0.1
Connecting to host 10.1.0.2, port 5201
[  5] local 10.60.0.1 port 36035 connected to 10.1.0.2 port 5201
[ ID] Interval           Transfer     Bitrate         Retr  Cwnd
[  5]   0.00-1.00   sec  2.59 MBytes  21.8 Mbits/sec  203   1.32 KBytes
[  5]   1.00-2.00   sec  1.05 MBytes  8.80 Mbits/sec  114   15.8 KBytes
[  5]   2.00-3.00   sec  1.48 MBytes  12.4 Mbits/sec  155   35.5 KBytes
[  5]   3.00-4.00   sec   758 KBytes  6.22 Mbits/sec  126   5.27 KBytes
[  5]   4.00-5.00   sec  1.60 MBytes  13.5 Mbits/sec  163   23.7 KBytes
[  5]   5.00-6.00   sec   948 KBytes  7.76 Mbits/sec  104   2.63 KBytes
[  5]   6.00-7.00   sec  1.23 MBytes  10.4 Mbits/sec  140   14.5 KBytes
[  5]   7.00-8.00   sec   948 KBytes  7.77 Mbits/sec  139   15.8 KBytes
[  5]   8.00-9.00   sec  1.36 MBytes  11.3 Mbits/sec  142   14.5 KBytes
[  5]   9.00-10.00  sec   948 KBytes  7.79 Mbits/sec  113   13.2 KBytes
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.00  sec  12.8 MBytes  10.8 Mbits/sec  1399             sender
[  5]   0.00-10.03  sec  12.4 MBytes  10.4 Mbits/sec                   receiver

iperf Done.
ubuntu@ubuntu-VirtualBox:~/5GLab/netns5G$ sudo ip netns exec ue11 iperf3 -c 10.1.0.2 -R -B 10.60.0.1
Connecting to host 10.1.0.2, port 5201
Reverse mode, remote host 10.1.0.2 is sending
[  5] local 10.60.0.1 port 46013 connected to 10.1.0.2 port 5201
[ ID] Interval           Transfer     Bitrate
[  5]   0.00-1.00   sec  4.50 MBytes  37.6 Mbits/sec
[  5]   1.00-2.00   sec  2.21 MBytes  18.6 Mbits/sec
[  5]   2.00-3.00   sec  2.50 MBytes  21.0 Mbits/sec
[  5]   3.00-4.00   sec  1.97 MBytes  16.5 Mbits/sec
[  5]   4.00-5.00   sec  2.20 MBytes  18.4 Mbits/sec
[  5]   5.00-6.00   sec  2.24 MBytes  18.8 Mbits/sec
[  5]   6.00-7.00   sec  2.30 MBytes  19.3 Mbits/sec
[  5]   7.00-8.00   sec  2.32 MBytes  19.4 Mbits/sec
[  5]   8.00-9.00   sec  2.32 MBytes  19.5 Mbits/sec
[  5]   9.00-10.00  sec  2.48 MBytes  20.8 Mbits/sec
- - - - - - - - - - - - - - - - - - - - - - - - -
[ ID] Interval           Transfer     Bitrate         Retr
[  5]   0.00-10.06  sec  25.3 MBytes  21.1 Mbits/sec  2164             sender
[  5]   0.00-10.00  sec  25.0 MBytes  21.0 Mbits/sec                   receiver

iperf Done.
ubuntu@ubuntu-VirtualBox:~/5GLab/netns5G$
```

50) Repeat previous measurements with the other two UEs (UE12 and UE21)

a. Compare the obtained results with the configured UL/DL AMBR for the three UEs

8. Slicing

51) Stop UE12 (Ctrl-C)

52) Restart UE12, now in the second slice (SST/SD: 2/112233) with a new configuration file and check the results

```
~/5GLab/netns5G$ ./UE12start-sl2.sh
(check the contents of file ./config/free5gc-ue12-sl2.yaml)
```

53) Observe the newly assigned IP address; what are the changes? Remember the observation done to the SMF yaml configuration file regarding the IP pools definition for the two defined slices

54) Make a ping from UE11 to UE12, now in different slices and observe the exchanged packets at the UPF; Is there connectivity?

   a. Check routing at the UPF namespace

      ```
      $ sudo ip netns exec upf ip route
      ```

   b. Add a new route in the UPF namespace

      ```
      $ sudo ip netns exec upf ip route add 10.61.0.0./24 dev upfgtp
      ```

55) Repeat the ping above.

9. Stop and reset the environment

56) Stop the UEs (Ctrl-C)

   a. Observe the messages generated by the gNBs (go to the respective terminal window)

57) Stop the gNB nodes (Ctrl-C)

   a. Observe the messages generated by the core entities (go to the respective terminal window)

58) Go again to the Web Console and check the 'Realtime Status' information

59) Stop the 5G Core in this order (Ctrl-C)

60) Wait for final processes to close (this takes some seconds, ending with "NRF terminated")

61) Delete the namespaces

```
~/5GLab/netns5G$ sudo ./5Gcleanup.sh
```

62) Power off the VM

Annexes

A. 5G System architecture

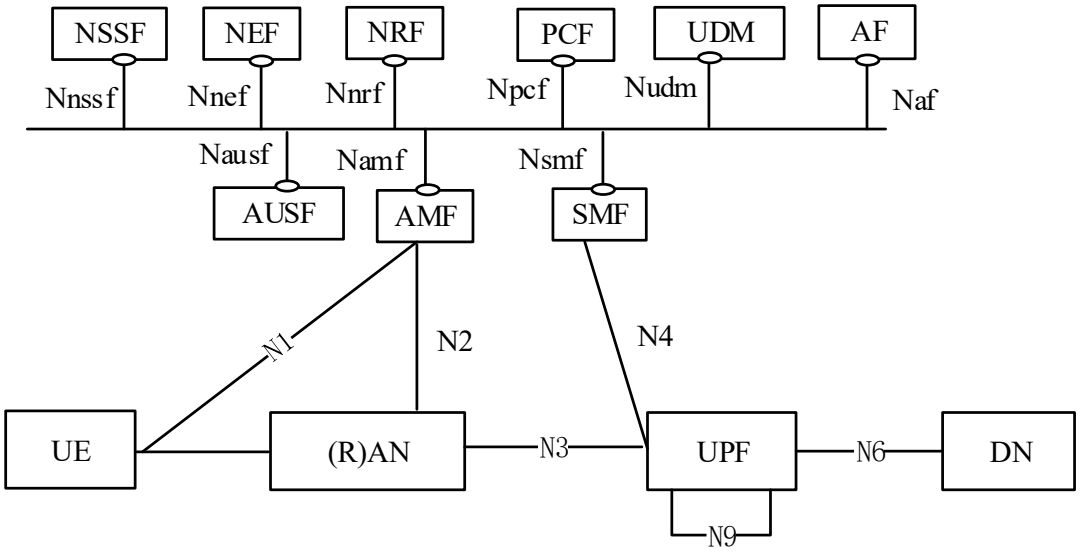


Figure 16: 5G Core, RAN and UE reference system architecture [3gpp]

B. Example procedure

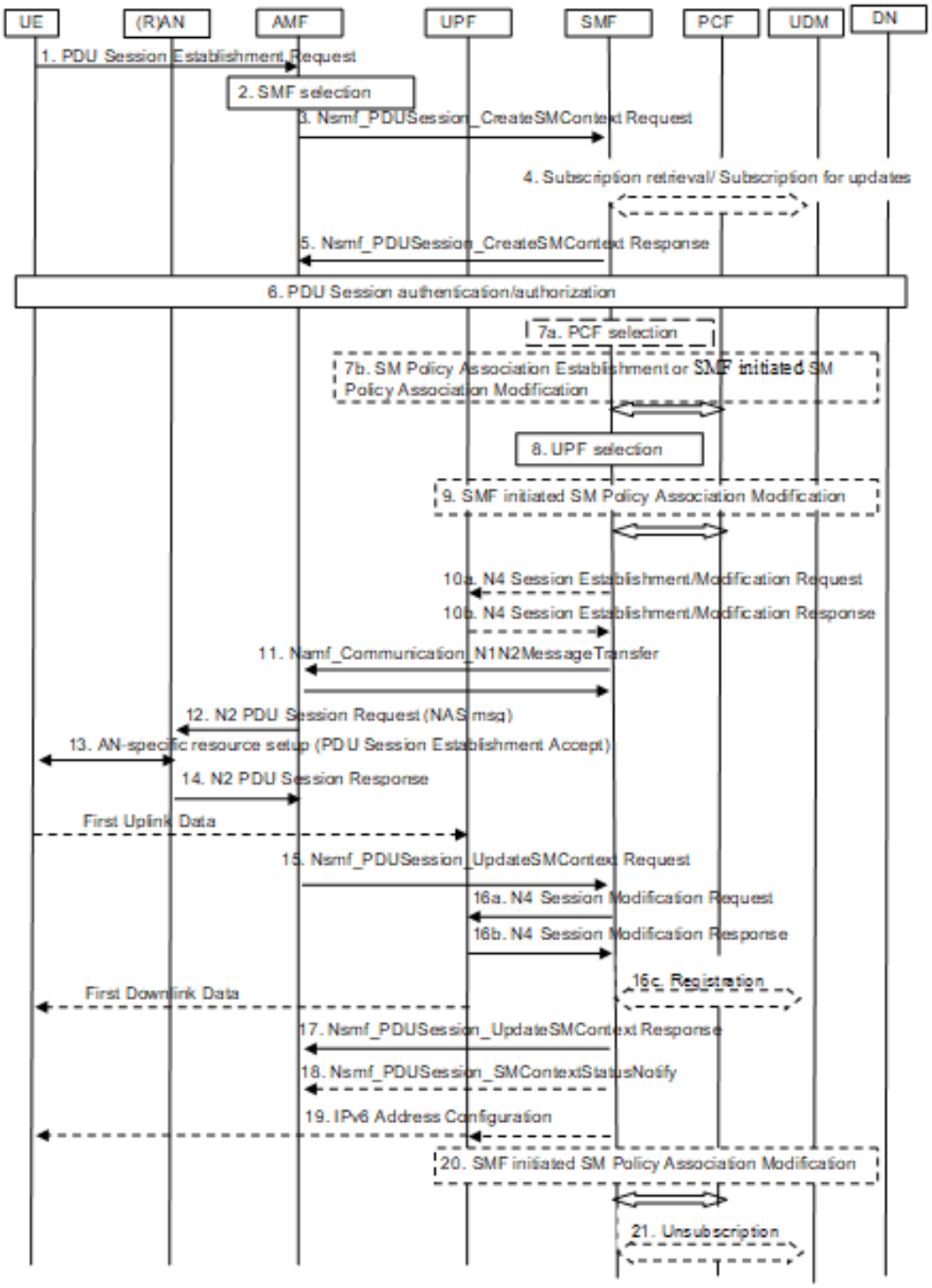


Figure 4.3.2.2.1-1: UE-requested PDU Session Establishment for non-roaming and roaming with local breakout

C. 5G Protocol stacks

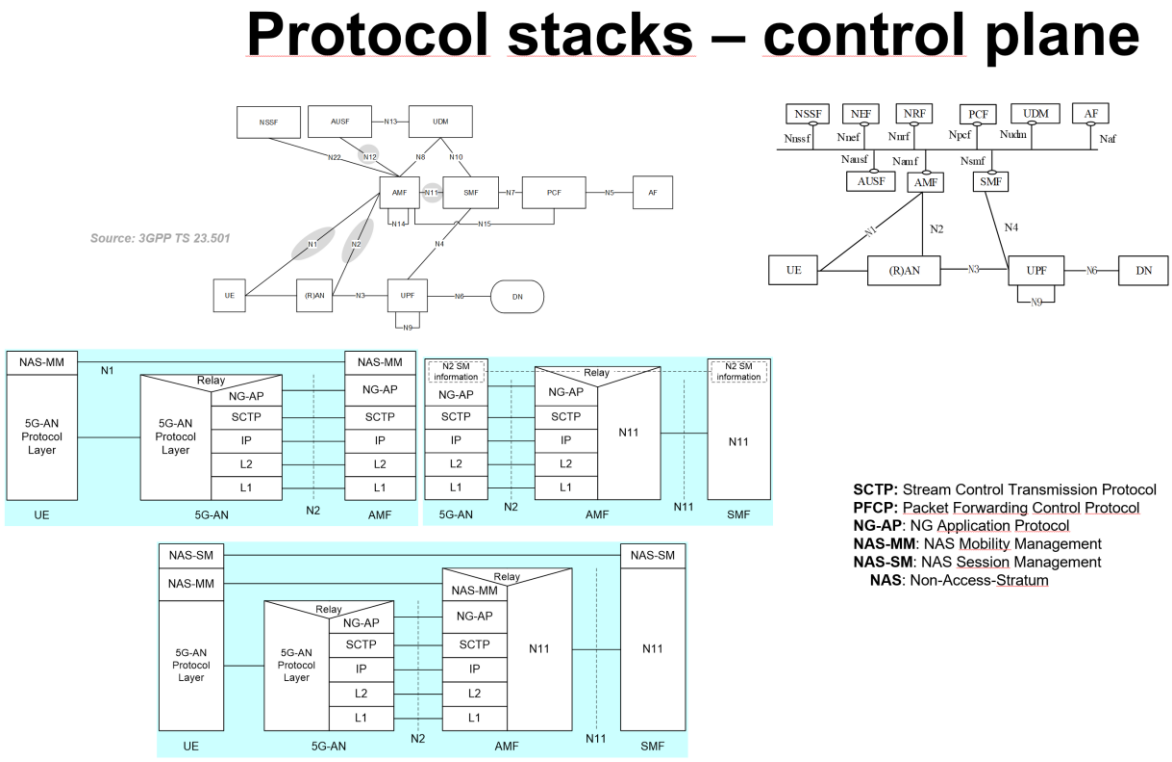


Figure 17: Protocol stacks, control plane

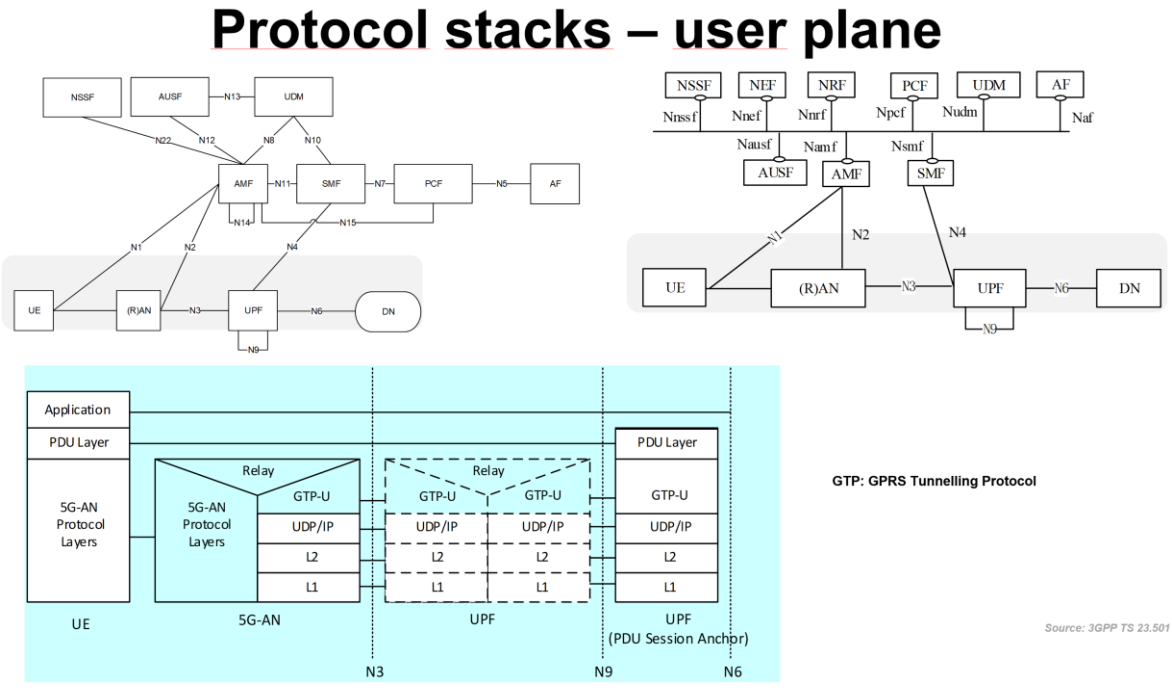


Figure 18: Protocol stacks, user plane

D. Free5GC New Subscriber creation form (UE11)

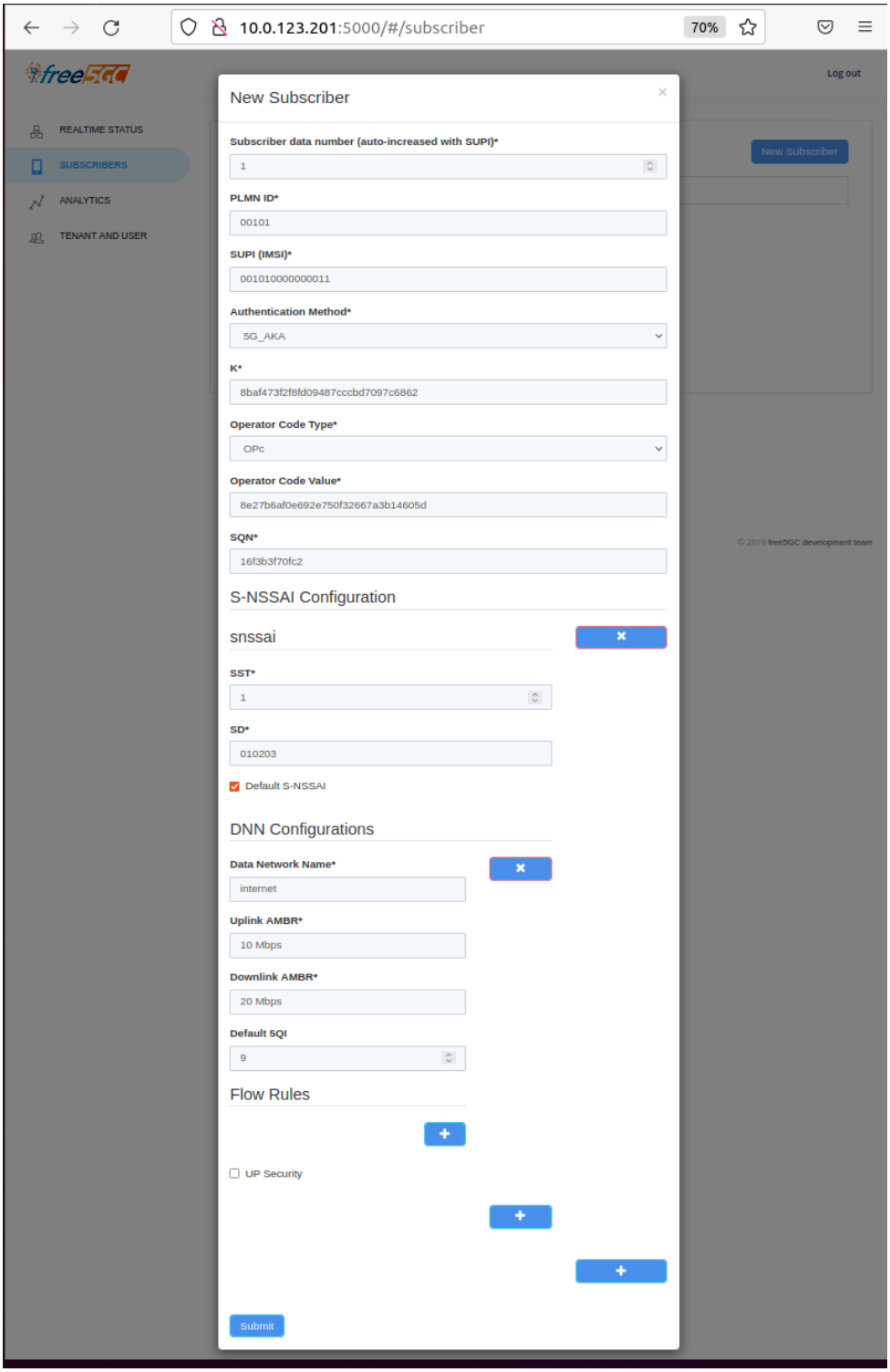


Figure 19: Example of a subscriber provisioning at Free5GC (UE11)

E. Example of a capture with Wireshark (with addresses resolution and display filter)

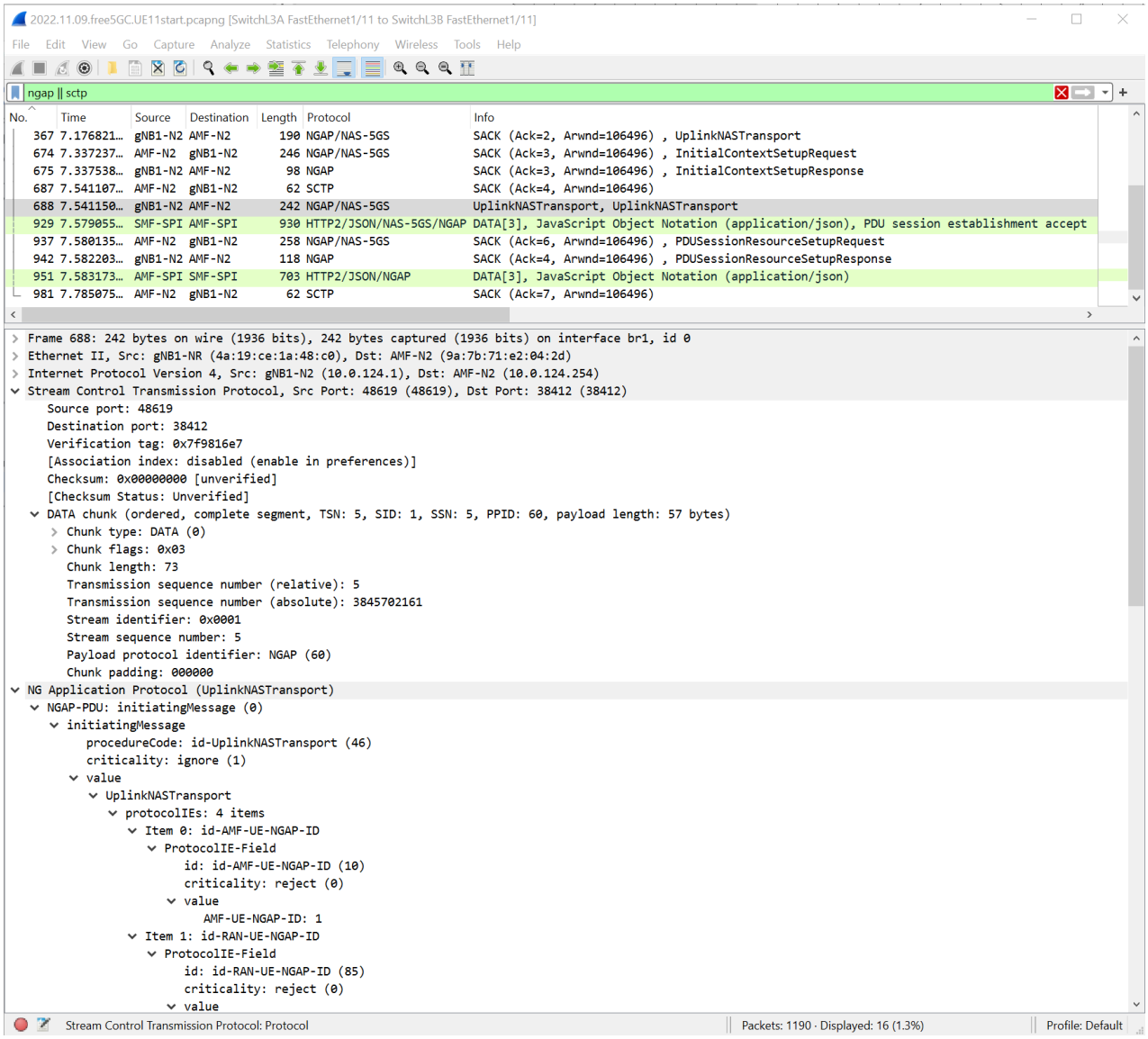


Figure 20: Example of a 5G exchange of messages (gNB – AMF interactions)



Figure 21: Detail of a 5G exchanged message (AMF and gNB via N2 interface)

F. Hosts file

```
#5G Core
10.0.123.1    NRF-SBI
10.0.123.2    UDR-SBI
10.0.123.3    UDM-SBI
10.0.123.4    AUSF-SBI
10.0.123.5    NSSF-SBI
10.0.123.6    AMF-SBI
10.0.123.7    PCF-SBI
10.0.123.9    SMF-SBI
10.0.123.100  MongoDB-SBI
10.0.123.200  hostm
10.0.123.201  WebConsole

10.0.124.254  AMF-N2
10.0.124.1    gNB1-N2
10.0.124.2    gNB2-N2

10.0.140.2    SMF-N4
10.0.140.1    UPF-N4

#5G dataplane
10.1.0.1      UPF-N6
10.1.0.1      Host-N6

#RAN1
10.0.201.1    UE11-NR
10.0.201.2    UE12-NR
10.0.201.254  gNB1-NR

#RAN2
10.0.202.1    UE11-NR
10.0.202.254  gNB1-NR
```

G. GTP (reinstalation)

In the situation at 5GCore start gtp fail with similar errors to these ones:

```
2025-11-04T20:51:00Z [FATA][UPF][Main] panic: runtime error: invalid memory address
or nil pointer dereference

2025-11-04T20:51:11Z  [ERRO][SMF][App]  Failed  to  setup  an  association  with
UPF[10.0.140.1], error:Request Transaction [1]: retry-out
```

1. Stop the 5gCore (Ctrl-C)
2. go to *~/5GLab/gtp5g* and issue the following commands
3. *make clean && make*
4. *sudo make install*
5. restart the 5GCore

H. Useful links

A. Free5GC:

a. [free5Gcore] https://www.free5gc.org/

b. [free5gcwiki] https://github.com/free5gc/free5gc/wiki

c. [konrad] https://github.com/konradkar2/netns5g

B. UERANSIM:

a. [ueransim] https://github.com/aligungr/UERANSIM/wiki

C. 3GPP

a. [3gpp] https://www.3gpp.org